

李乐岩

3073751449@qq.com | +86 18805506898 | GitHub: <https://github.com/leyancode>

爱丁堡大学 **University of Edinburgh** 英国 | 2025.09 – 2026.11
高性能计算与数据科学 硕士
课程与毕业论文 2026.08 完成 | 可入职时间 2026.09.01 | 学位证书 2026.11 获得
利物浦大学 **University of Liverpool** 英国 | 2023.09 – 2025.06
计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

项目经历:

面向 HPC Benchmark 的 Agent 式性能分析系统 (GitHub 开源、可复现)

项目描述: 基于真实 ARCHER2 超算上的 PETSc benchmark 运行数据, 构建"确定性工具 + 可评测路由"的 Agent 式性能分析系统: 将 benchmark 结果结构化为 SQLite 数据库, 把性能分析任务封装为确定性分析工具, 用中文规则 router 作为可验证 baseline, 通过 CLI 以中文完成最优配置查询、模式对比与扩展性报告生成; 进一步接入 LLM function calling router, 用自建 30 问五类中文评测集对两种 router 打分——规则 83.3%、LLM 100%, 并由消融实验定位分数主要来源为 system prompt 而非模型本身。代码与评测集开源: github.com/leyancode/hpc-benchmark-agent

技术栈:

Python 、 **SQLite** 、 **pytest** 、 **OpenAI SDK** (**function calling**) 、 **PETSc**、 **MPI**、 **OpenMP**、 **Slurm**、 **Git**、 **Linux**

工作内容:

- 设计分层架构: 数据导入层 (CSV → SQLite)、指标层 (speedup / parallel efficiency)、工具层 (最优配置查询、模式对比、扩展性报告)、路由层 (规则与 LLM function calling 双实现、输出严格同构)、可观测层 (trace_id + 事件流) 与 CLI 层; 各层输入输出合同由 165 项自动化测试钉死, 全量测试离线可跑。
- 实现中文规则 router: 关键词意图分类 + 问题规模解析 (阿拉伯数字、"100 万"混写、中文数字"一百万"), 缺参数主动追问、超范围明确拒答; 输出与 LLM tool call 严格同构 (intent + 参数), 两种 router 因此可共用一套评测集, CLI 上切换只需改一行 dispatch。
- 封装确定性分析工具: 单规模最优配置查询、MPI-only 与 hybrid 模式对比 (数据只含 ranks/threads, 故采用保守标签 mpi_like / hybrid_like)、扩展性报告生成; 同一数据库生成的 Markdown 报告字节级一致 (刻意不含时间戳), 使"文档是否漂移"退化为一次 git diff 检查。

- 基于真实 ARCHER2 数据验证：100 万未知数规模下单核 245.7s → 128 核 3.866s (speedup 63.6、并行效率 49.7%)，识别出 2/4 核超线性加速 (cache 效应，效率 147% / 126%) 与 32 核 hybrid 配置的效率坍塌 (0.33)，并由规则自动生成中文解读段落。
 - 接入 LLM function calling router：设计 tool schema (description 写明工具边界；unknowns 刻意可选——模型省略参数即「追问」、不调用工具即「拒答」)；实现防御式响应解析，把未调用工具、幻觉工具名、arguments 非合法 JSON 三类异常统一收敛为显式 unknown，绝不猜参数以避免「静默算错」；API 失败向上抛出而非回退规则 router，以免污染评测。单元测试全部经 FakeLLM 依赖注入离线运行，无需 API key。
 - 建立评测机制与对比结论：自建 30 问五类中文评测集 (正常 / 追问 / 拒答 / 数字陷阱 / 规则已知会错的对抗题)，期望值为 tool call 而非答案文本，harness 接受任意 router 函数、对其实现无知；规则 router 83.3%、LLM router 100%，而消融实验显示同一模型不加 system prompt 仅 63.3%——准确率主要由工具说明书与策略 prompt 决定；报告如实标注局限 (题量 30、单模型单次运行)。
 - 实现结构化 tracing：每次提问生成唯一 trace_id，路由决策与工具调用以 JSON Lines 事件流记录 (ask_start → route_result → tool_call / tool_result → answer)，可完整回放一次 agent 执行；脱敏收敛在单一出口点 (绝对路径降级为文件名、疑似密钥字段一律 redacted)，并由扫描测试强制保证。
 - 工程纪律：pytest 全量离线可跑、不依赖 API key (key 走环境变量，永不入库)；README demo 命令逐条可复现 (已用全新 clone 从零验证)；以"公开声称 == 已实现功能"作为文档与简历的验收约束。
-

多智能体 Actor-Critic 方法中的价值函数分解研究

该项目研究将 VDN、QMIX 等价值函数分解方法引入 MAPPO 框架，探索 factorised critic 在 cooperative multi-agent environments 中对协作效率、收敛稳定性和策略表现的影响。该项目可作为 AI Agent 方向中多智能体协作、任务分解和策略学习的算法基础展示。代码开源：github.com/leyancode/marl-value-factorisation

技术栈：

Python、PyTorch、ePyMARL、PyMARL、MAPPO、VDN、QMIX、多智能体强化学习

工作内容：

- 基于 ePyMARL / PyMARL 框架实现 MAPPO、MAPPO-VDN、MAPPO-QMIX 等多智能体强化学习算法变体，

并完成训练配置、实验运行和结果评估。

- 修改 actor-critic 框架中的 critic 网络结构，引入 VDN 风格的加性价值分解和 QMIX 风格的单调混合网络，探索 factorised critic 在 cooperative multi-agent tasks 中的适用性。
 - 支持 centralized training with decentralized execution 训练范式，在训练阶段利用全局状态和联合动作信息，在执行阶段由各 agent 基于局部观测独立决策。
 - 在 Matrix Game 和 Predator-Prey 环境中设计并完成对比实验，评估不同算法在 test return、收敛速度、最终回报和协作表现上的差异。
 - 构建实验日志记录与结果分析流程，对 reward curve、test return、convergence behaviour 和 training stability 等指标进行可视化和比较。
 - 分析价值函数分解方法应用于 actor-critic 框架时可能出现的不稳定问题，包括 critic approximation error、多智能体非平稳性、credit assignment 和 mixing network 表达能力限制。
 - 完成完整学术论文撰写，系统整理算法设计、代码实现、实验结果、局限性与后续改进方向。
-

技术能力

编程语言与工程开发： 熟悉 Python、C/C++，掌握 SQL 基础，了解 JavaScript、HTML、CSS；可用 Python 完成 AI 应用原型、数据处理、日志解析、实验自动化与模型训练流程。

AI Agent 与 LLM 应用： 了解 LLM Agent 基本架构，包括 tool calling、任务分解、RAG 检索增强生成、prompt engineering、记忆管理、多轮对话状态管理和 Markdown 报告生成；具备将大模型能力与外部工具、数据库和自动化脚本结合、并对其做离线评测的项目开发经验。

RAG 与向量检索： 了解 Markdown 文档解析、文本切分、embedding 生成、FAISS / Chroma 向量索引、top-k 语义检索和上下文拼接流程；理解 RAG 在领域知识库问答中的适用场景与局限——本项目的数据源是结构化 benchmark 数据库，SQL 查询在准确性与可验证性上优于向量检索，因此未引入。

后端与系统设计基础： 了解 FastAPI、REST API、HTTP、客户端与服务端交互、参数校验、异常处理、日志追踪、TraceID、任务调度、限流、幂等性和可观测性等后端工程概念。

数据库与缓存： 了解 MySQL / SQLite 使用及索引、事务、锁、查询优化等概念（本项目中实际设计了 benchmark 表结构与查询索引）；了解 Redis 常用缓存场景与 TTL、缓存穿透 / 击穿、一致性问题。

机器学习与数据分析： 熟悉 PyTorch，具备深度学习和多智能体强化学习算法实现与实验评估经验；熟悉 Pandas、NumPy、Matplotlib，可完成结构化数据处理、指标计算、实验日志分析和可视化。

工具与开发环境： 熟悉 Linux、Bash、Git、VS Code、Docker 基础操作；具备用自动化脚本管理实验、处理日志并构建可复现开发流程的经验。

论文发表

Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis.
International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.